



智测星图项目问题回答

项目整体流程详解

```
pythonproject/
├── base/                                # 基础类封装
│   ├── apiutil.py                      # 核心请求处理类
│   ├── apiutil_business.py             # 业务场景处理类
│   ├── generateId.py                   # 生成测试ID
│   └── new_testcase_tools.py            # 测试用例生成工具
├── common/                              # 公共方法封装
│   ├── sendrequest.py                  # HTTP请求封装
│   ├── readyaml.py                     # YAML数据读写
│   ├── assertions.py                   # 断言工具类
│   ├── debugtalk.py                    # 自定义函数库
│   ├── connection.py                   # 数据库连接
│   ├── recordlog.py                    # 日志记录
│   └── dingRobot.py                    # 钉钉机器人通知
├── conf/                                # 配置文件目录
│   ├── config.ini                      # 环境配置(API地址、数据库配置等)
│   ├── operationConfig.py              # 配置文件读取类
│   └── setting.py                       # 全局配置变量
├── data/                                # 测试数据目录
│   └── loginName.yaml                  # 数据文件
├── testcase/                            # 测试用例目录
│   ├── Single interface/               # 单接口测试
│   │   ├── test_debug_api.py           # 测试脚本
│   │   └── addUser.yaml                 # 测试数据
│   └── Business interface/             # 业务场景测试
│       └── ProductManager/             # 产品管理模块测试
├── report/                              # 测试报告目录
│   ├── allureReport/                   # Allure报告
│   └── temp/                           # 临时数据
├── logs/                                # 日志目录
├── extract.yaml                          # 接口依赖参数存储文件
├── conftest.py                           # Pytest全局配置(fixture)
├── pytest.ini                           # Pytest配置文件
├── requirements.txt                      # 依赖包列表
└── run.py                               # 主程序入口
```

`testcase` 文件中的 **YAML** 是用来存储接口的**基本信息**（`baseInfo`：地址、方式、请求头）、**测试数据**和**预期结果**（`testCase`）

1. 一个YAML文件的 `testCase` 下可以有多个测试用例（正向、逆向用例都能写到此处：框架会为**每一个用例生成一个独立的测试**）
2. 请求头header可以根据**接口类型**来选择：
 - 表单提交： `application/x-www-form-urlencoded`
 - JSON提交： `application/json`
 - 文件上传： `multipart/form-data`
3. **请求参数**同样也有三种类型： `data`、`json`、`params`
 - GET请求必须用 `params`，不能用 `data`： `params` 参数是拼接在URL后面的（GET请求没有请求体）
 - `data` 参数是放在**请求体**里的
4. `validation` 部分：代表的是**断言**，用来检验响应结果**是否包含某个值**，一起有五种
 - `contains` - 包含断言（最常用）：只要包含有指定的字段即可通过，不需要字段一模一样
 - `eq` - 相等断言（精确匹配）：需要指定的字段一模一样才能通过
 - `ne` - 不相等断言：不等于指定字段时通过
 - `rv` - 响应结果任意值断言：检查响应JSON里任意位置的某个字段，不常用
 - `db` - 数据库断言（高级）：验证数据库里的数据是否正确（在MySQL数据库执行这里写的SQL语句，如果能查到数据，断言通过）

`testcase` 文件中的 **Python文件** 是测试脚本

1. `@pytest.mark.parametrize`：数据驱动，用来从写好的YAML文件中来读取数据
2. 定义好的方法是用来执行测试
 - `base_info`：YAML的`baseInfo`部分
 - `testcase`：YAML的每一个`testCase`
 - `RequestBase().specification_yaml()`：发送请求并验证结果

接口关联部分（参数传递）—最核心的部分

在实际测试中，很多**接口**有**依赖关系**：

- 场景1：登录 → 其他操作：登录就需要获取当前用户token
- 场景2：新增 → 查询/删除：要查询/删除/新增哪个用户：需要知道用户的`user_id`

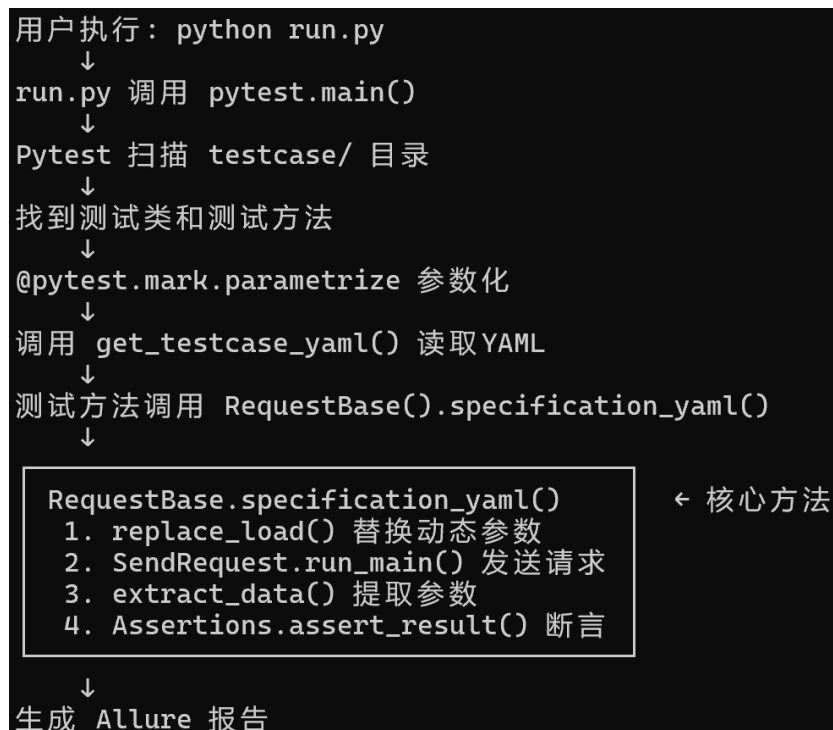
问题来了：这个token、user_id等参数怎么从上一个接口传给下一个接口？

解决方案：框架使用一个**中转文件** `extract.yaml` 来存储接口之间传递的参数



- 在接口的YAML文件中，利用 `extract` 关键字（还有 `extract_list` - 提取多个值）从响应结果中提取参数再写入 `extract.yaml`，`$.data.token`（表示 `data` 下的 `token` 字段）是 JsonPath表达式，表示从响应JSON里提取 `data.token`
- 在其他接口中可利用 `${get_extract_data(token)}` 来读取 `extract.yaml` 里的 `token` 字段

框架核心执行流程（源码分析）



核心类1：RequestBase 类（整个框架的心脏，所有请求都经过它处理）

1. `specification_yaml()` 方法（核心的步骤，再往下面介绍的方法和核心类都是为了此方法而服务的）：将YAML里的数据进行动态的拼接成可以发送的HTTP请求数据，并给你们发送给后端，然后提取出响应结果给到 `extract.yaml`，最后验证响应结果是否符合预期并生成测试报告
 - 注意：YAML里的配置都是以键值对的形式，因此在代码中涉及很多键值对的操作
2. `replace_load()` 方法：参数替换的核心，把YAML文件里的 `${函数名(参数)}` 替换成真实的值
 - 核心：反射函数 `getattr()`，作用是动态获取对象的属性或方法（`getattr(对象, '属性名/方法名')`）
 - 使用 `getattr()` 函数动态调用 `DebugTalk` 类的方法
3. `extract_data()` 方法：把响应数据写入 `extract.yaml`

核心类2：SendRequest 类（负责发送HTTP请求）

1. `send_request()` 方法：统一的请求发送方法
 - `session.request(**kwargs)` 是核心，支持所有HTTP方法
 - 自动提取Cookie并保存到 `extract.yaml`
 - 异常处理：网络异常会自动记录日志并失败

核心类3：Assertions 类（负责断言验证）

全局配置： `conftest.py`

1. 定义了一个session级别的 fixture，在所有测试开始前自动执行，用来清空 `extract.yaml` 文件和清理旧的测试报告。这样可以确保每次测试都是从干净的状态开始，避免旧数据干扰。
 - 否则在所有测试用例中都要添加清空 `extract.yaml` 文件和清理旧的测试报告和连接数据库的代码
2. 定义了一个钩子函数 `pytest_terminal_summary`，在所有测试结束后自动触发，收集测试统计信息（通过数、失败数、总时长等），然后发送钉钉通知给团队

项目面试高频题以及答案

项目遇到问题以及解决方法-token过期

我遇到了接口自动化中token过期的问题，就是我的token过期了导致自动化失败

请求前检查

这种方法在发送实际业务请求前，先检查当前Token是否即将过期。

实现方式：

1. 在登录后，不仅保存 `token` 本身，还要记录 Token 的有效期 (`expires_in`) 和 获取Token的时间戳 (`login_time`)。
2. 在发送任何需要认证的请求之前，计算当前Token是否已接近过期（例如，设定一个缓冲时间，如提前5分钟刷新）。
3. 如果快过期了，则先调用刷新Token的接口获取新Token，再用新Token发送请求。

优点：避免了大部分因过期导致的请求失败。

缺点：增加了每次请求前的判断逻辑；如果Token失效时间突然改变，缓冲时间设置不合理可能失效。

请求后检查

这种方法是在收到请求失败的响应后，判断失败原因是否为Token过期。如果是，则自动刷新Token并重试原请求。

实现方式：

1. 对所有出站的请求进行封装（通常使用 请求会话对象 (Session) 或 装饰器/钩子函数）。
2. 在收到响应后，检查状态码和响应体，判断是否为Token过期错误。
3. 如果是，触发Token刷新流程，然后用新Token重新发送刚才失败的请求。
4. 重试次数通常设为1次，避免无限循环。

优点：无需预测Token过期时间，逻辑更通用。

缺点：第一次请求必然会失败，虽然对用户无感，但会产生一条错误日志。

项目遇到问题以及解决方法-如何解决的-YAML模板解析

YAML模板解析问题：当包含复杂数据结构时解析失败

解决方案：引入ruamel.yaml替代PyYAML，支持更灵活的语法

动态参数传递：需要支持函数调用和变量替换

解决方案：开发了`${}`语法解析器，结合正则表达式实现参数替换

Allure报告环境变量丢失：

解决方案：将environment.xml放在根目录，运行后复制到报告目录

可以说说项目的整个流程是什么样子的吗

红色部分是重点

1. 每个测试方法通过`get_testcase_yaml()`从YAML文件加载测试数据（首先是加载测试数据）

2. 使用@pytest.mark.parametrize实现数据驱动测试
3. 通过@pytest.mark.run(order=x)控制测试执行顺序，其中order是排序的次序（然后就是根据顺序开始执行case）
4. 使用RequestBase().specification_yaml()执行实际的API请求（然后去发送请求）
5. 在代码里面我们可以去断言一些我们的校验点
6. 最后，Allure框架会生成测试报告

口语版本的流程介绍

这是一个基于Python的接口自动化测试框架，框架采用Pytest作为测试引擎，结合Allure生成可视化报告，支持YAML格式的测试用例管理。项目的整体架构分为common公共组件，包括日志记录，断言，请求发送，AML读写操作这些。conf里面放一些配置，base里面放一些接口处理，详细的测试用例在testcases里面。

整体的执行流程就是：

1. 读取YAML用例：readyaml.py读取测试用例文件
2. 请求预处理：处理一些参数变量等等
3. 发送请求：通过封装的requests库，支持各种HTTP方法和参数类型
4. 响应处理：结果提取：支持正则和jsonpath两种方式，assertions.py处理各种断言逻辑
5. 最后集成allure报告，并且记录请求和响应的信息

说说你这个项目自动化测试框架的整体架构

该框架采用分层设计模式，主要包含：

基础层：封装HTTP请求模块，支持GET/POST/PUT/DELETE等请求方式

数据层：使用YAML文件管理测试用例，实现数据与代码分离

工具层：开发了参数提取(jsonpath/正则)、函数调用等核心功能

报告层：集成Allure生成可视化测试报告

做了多少接口测试

在该项目中：（下面的数据大家悠着一点说，可以自己补充补充）

已完成10个核心接口的自动化测试覆盖

编写了30个测试用例（平均每个接口3个用例）

覆盖了5种请求方式（GET/POST/PUT/DELETE/文件上传）

包含3种业务场景（用户认证、数据查询、文件处理）

接口自动化流程

1. 测试准备阶段

测试用例设计：为每个接口创建独立的YAML文件（如addUser.yaml），文件中包含：

接口基本信息（URL、请求方法、请求头等）多个测试场景（如正常新增、重复新增、参数缺失等）
每个场景的输入数据和预期结果

测试数据准备：使用独立的CSV/Excel文件管理测试数据,通过extract.yaml存储需要跨用例共享的参数（如用户ID）

2. 测试执行阶段

以"新增用户"接口为例：

读取YAML文件：系统自动加载addUser.yaml中的测试场景

动态参数处理：自动生成测试需要的随机数据（如用户名）从共享文件获取依赖参数（如权限token）

发送请求：按照YAML中的配置向目标URL发送HTTP请求

结果验证：检查响应状态码是否符合预期,验证返回数据中的关键字段,必要时查询数据库确认数据变更

2. 测试报告阶段

自动记录：每个测试步骤的执行详情都会被记录

包括请求内容、响应数据、验证结果自动截取关键节点数据

生成报告：将所有测试结果整理成可视化网页,按模块/接口分类展示,用不同颜色标识通过/失败的用例

问题定位：失败用例会显示详细的错误上下文,可查看请求与响应的完整对比,附带相关的数据库查询结果

2. 测试维护

用例更新：直接修改YAML文件即可调整测试逻辑

数据更新：维护外部数据文件实现批量修改

执行控制：可通过标记选择执行特定模块,支持失败用例重试机制,可以灵活设置执行顺序

yaml是怎么管理测试数据的

在我们的测试框架中，YAML测试数据管理就像用标准化表格管理测试用例：

1. 文件组织方式

每个功能模块有独立文件夹（如"用户管理"）

每个接口对应一个YAML文件（新增用户、查询用户等）

测试数据与代码完全分离，像填Excel表格一样维护用例

2. YAML文件结构 每个文件包含两大模块：

接口基本信息：地址、请求方式、固定请求头等

测试场景集合：包含多个测试用例，每个用例有：场景描述（如"密码错误测试"），输入参数（支持自动生成随机数据）预期结果（多种验证方式）

2. 核心特点

易读易改：像文档一样直观，非技术人员也能维护

动态灵活：支持自动生成测试数据、跨用例参数传递

执行有序：可以控制测试用例的执行顺序

报告友好：每个用例自动生成详细测试报告

2. 实际效果

测试人员只需编辑YAML文件即可新增用例,修改测试数据无需改动代码,自动生成美观的测试报告，直观展示每个用例的执行情况

allure是怎么生成报告的

1. 安装依赖：pip install allure-pytest # Python插件
2. 运行测试并生成数据：pytest --alluredir=./result # 测试结果存到./result目录
3. 生成报告：allure serve ./result # 自动打开浏览器看报告
4. 查看报告：allure open ./report

项目的断言机制如何构建

此项目采用了分层设计，断言逻辑被封装在 common.assertions.Assertions 类中（通过 self.asserts.assert_result() 调用）

为什么要这样做：

- 断言逻辑集中管理（修改断言策略只需改 Assertions 类）
- 支持复杂的断言规则（如数据库校验、多字段组合校验）
- 断言失败会记录到日志和Allure报告

如何加载yaml文件

读取 YAML 文件

```
test_data = read_yaml('./data/register_success.yaml')
```

在 pytest 中使用

```
@pytest.mark.parametrize('data', read_yaml('./data/register_success.yaml'))  
  
def test_example(data):  
  
    print(data) # 这会输出 YAML 文件中的每一组数据
```

你们是怎么接入 Jenkins 的？具体配置了什么？

我们通过 Pipeline as Code 的方式接入 Jenkins。具体来说：

第一步：在 Jenkins 上安装必要插件，包括 Pipeline、Git、Allure Plugin 和邮件插件。

第二步：在项目根目录写一个 `Jenkinsfile`，里面定义了流水线阶段：

- 从 Git 拉取代码
- 执行 `pip install -r requirements.txt` 安装依赖
- 运行 `pytest --alluredir=./allure-results` 执行测试并生成报告数据
- 调用 Allure 插件生成可视化报告
- 发送邮件通知团队

第三步：配置触发器，包括 Git Webhook（代码推送自动触发）和 定时任务（每天凌晨2点回归）。

另外还配置了参数化构建：可以通过参数选择执行哪个模块的用例，或者切换测试环境。

具体配置了：Git 仓库地址和凭据、Python 环境、Allure 结果目录、邮件收件人和模板、失败重试次数（2次）、以及构建后的清理工作空间。

你的项目有重试机制

1. 选用插件：pytest 本身不带重试，我主要用 `pytest-rerunfailures` 这个插件来实现
2. **全局配置**：通过命令行或配置文件统一控制。例如：`pytest --reruns 3 --reruns-delay 1`（对失败用例重试 3 次，每次间隔 1 秒）。
3. **单用例定制**：对网络请求、数据库操作等不稳定场景，用 `@pytest.mark.flaky(reruns=2, reruns_delay=0.5)` 单独标记。
4. 条件重试：只对特定异常（如 `ConnectionError`）重试，其他失败立即终止，避免掩盖真正 bug。

另外，偶尔需要更灵活的控制时，我也会在用例内部用循环 + `try...except` 手工实现简单重试。整体上，重试机制有效提升了测试的稳定性，尤其是对抗环境抖动。